

It is an area or region within that we can access specific var & fun.

Lexical Environment

- When ever JS code run, it create GRC in memory it create lexical scope
- lexical scope is local memory & parent's lexical memory.
- **Lexical Memory = Local memory + Lexical Env of parent's All memory.**
- Lexical means Hierarchy (one inside another).

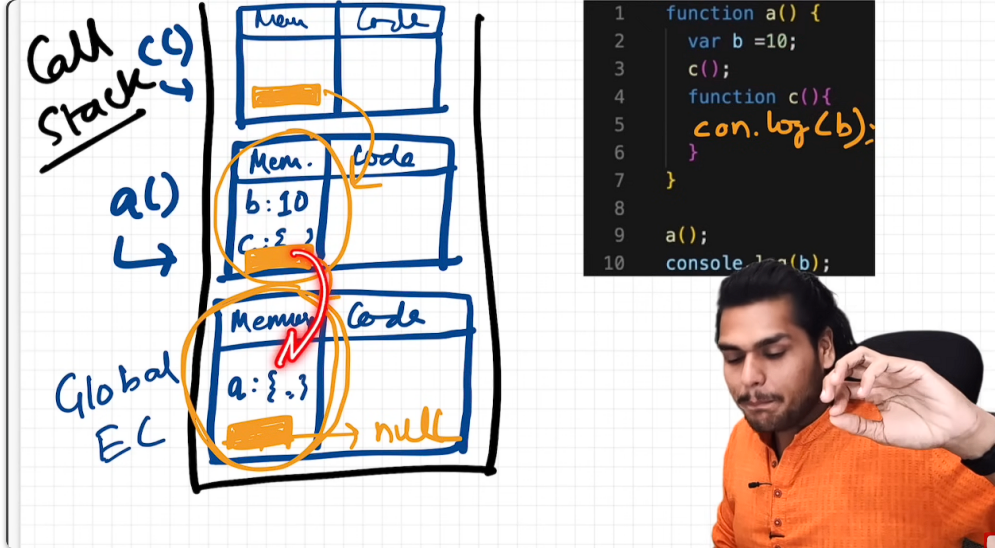
E.g. let's understand by e.g.

- Lexical meaning that one side another here, function `c` is inside function `a` lexically.
 - Here Function `c` get in FEC it get ref of their parent (lexical) memory means function `a` like wise function `a` get memory ref their parent current is GEC.

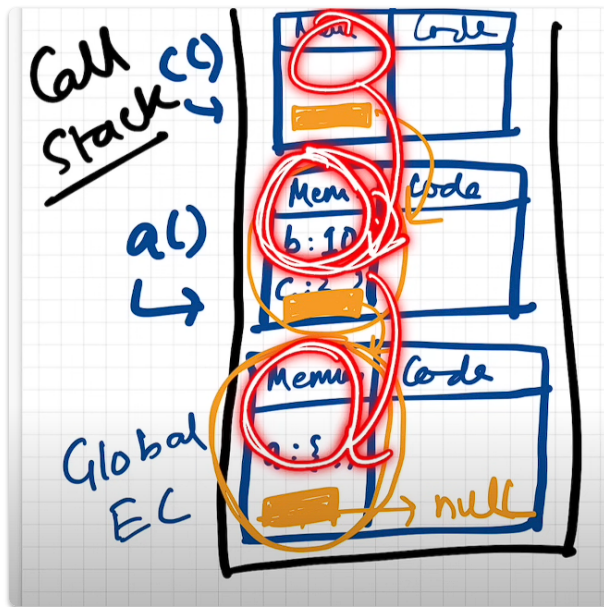
Case 1:

- Here, in function `c` when line `console.log(b)` first of it will look int their memory if found then access that variable else it goes to their parent memory if not their then here it will go to Global execution panel if not there then will throw error.
- Here, found inside function `b`.

```
function a() {  
  var b = 10;  
  c();  
  
  function c() {  
    console.log(b)  
  }  
}  
  
a();  
console.log(b);
```



- Scope chaining



Temporal Dead Zone

- **Temporal Dead Zone (TDZ)** refers to the time between the hoisting of a variable (using `let` or `const`) and its initialization. During this time, the variable cannot be accessed.
- `let` & `const` also are **Hoisted but diff from 'var'**.
- These are in temporal dead zone for that time.

```

console.log(b);
console.log(a); // Cannot acc
let a = 10;
var b = 100;

```

Pause on caught exceptions

function.js

- ☒ console.log(b);

Scope

Script

- a: <value unavailable>

Global

- ▶ alert: f alert()
- ▶ atob: f atob()
- b: undefined

- `var`: Hoisted and initialized with `undefined`.
- `let` and `const`: Hoisted but **not initialized**. They enter the TDZ until they are explicitly assigned a value.

```
console.log(a); // undefined (var is hoisted and initialized to undefined)
var a = 10;

console.log(b); // ReferenceError: Cannot access 'b' before initialization
let b = 20;

console.log(c); // ReferenceError: Cannot access 'c' before initialization
const c = 30;
```

1. Syntax Error

- **Definition:** A Syntax Error occurs when the code does not conform to the language's rules or syntax norms.
- **Characteristics:**
 - Indicates a problem with the structure of the code.
 - Prevents the code from being executed.

Examples:

1. Declaring a `const` without an initializer

```
const x; // SyntaxError: Missing initializer in const declaration
```

2. Function declaration without braces

```
function myFunction // SyntaxError: Function declarations require a function body
```

2. Reference Error

- **Definition:** A Reference Error occurs when the code tries to access a variable or function that is not defined or accessible in the current scope.
- **Characteristics:**
 - Indicates a problem with the variable's or function's availability.

Examples:

1. Accessing a variable before initialization

```
console.log(a); // ReferenceError: Cannot access 'a' before initialization
let a = 10;
```

2. Accessing a variable that is not defined

```
console.log(bb); // ReferenceError: bb is not defined
```

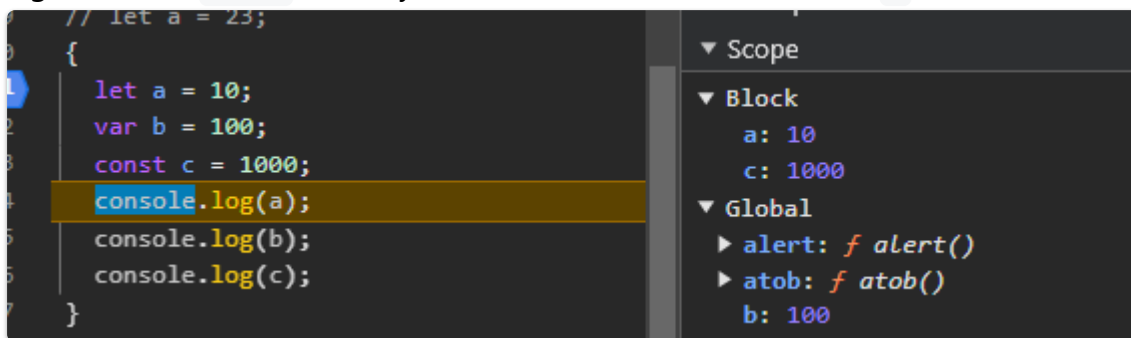
Block Scope

- Block Scope is defined this `{.....}`.
- It basically **grp all statement into one block** E.g. if else . Loop. Fun. Class inside this all consider as block.
- **Let & const are in Block scope.**
- For `let` and `const` separate new memory block will be created.

Examples:

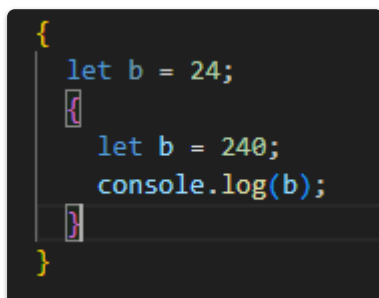
1. Declare Variable & Access within Scope

- `let` & `const` are **BLOCK Scope** so outside the block it won't be accessible.
- `var` is function-scoped — it's accessible throughout the function.
- `let` and `const` are block-scoped — only accessible within `{ }` where declared.
- E.g. Here, `var a & c` are only accessible within that block and `b` store store inside global scope.

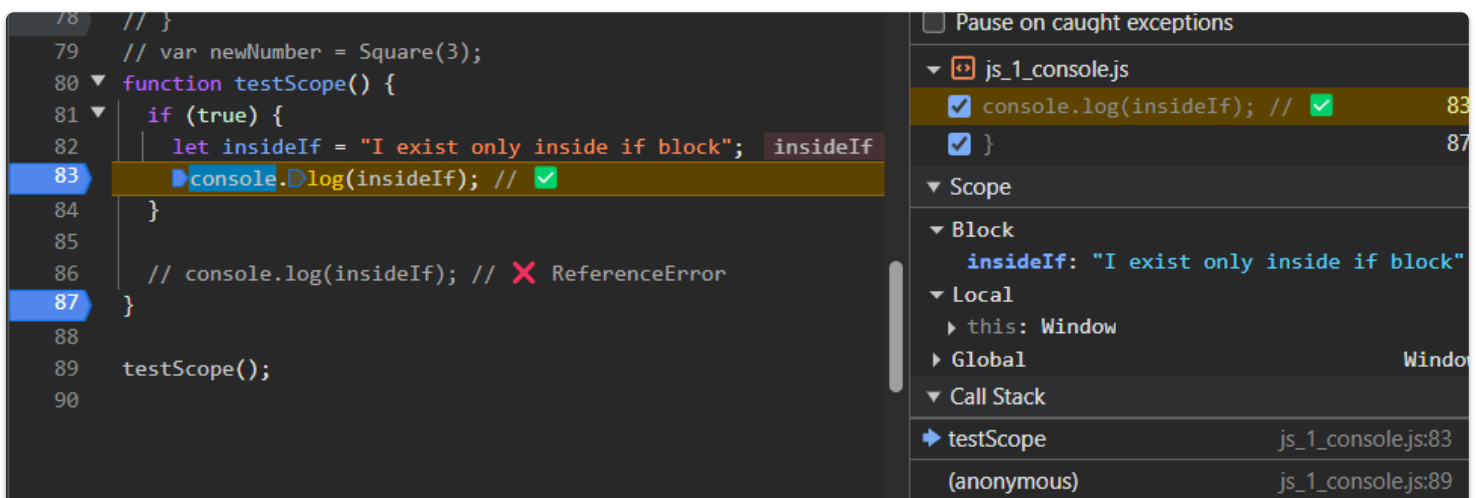


2. Nested Scope (Lexical Scope)

- Block Scope also Follow Lexical Scope like func.



3. Declaring variables inside `try`, `if`, `for`



Shadowing

- In that value get override(modify) to new value.

Examples:

1. Override the Value

- Variable declare using `var` gets override due to Global **Scope**

```
var c = 200;
let d = 3000
console.log(c);
{
  let a = 10;
  const b = 20;
  var c = 30;
  console.log(a);
  console.log(b);
  console.log(c);
}
console.log(c);
```

Scope

- Block
 - a: 10
 - b: 20
- Script
 - d: 3000
- Global
 - alert: f alert()
 - atob: f atob()
 - blur: f blur()
 - btoa: f btoa()
 - c: 30
 - caches: CacheStorage {}
 - cancelAnimationFrame: f cancelAnimationFrame()

```
// testScope();
function testVarShadowing() {
  var value = "Outer scope"; value = "Inner block scope"

  if (true) {
    var value = "Inner block scope"; value = "Inner block scope"
    console.log("Inside block:", value); // Outputs: Inner block scope
  }

  console.log("Outside block:", value); // Also outputs: Inner block scope
}

testVarShadowing();
```

var value = "Outer scope"; 91
var value = "Inner block scope"; 94

Scope

- Local
 - this: Window
 - value: "Inner block scope"
- Global
 - Window
- Call Stack
 - testVarShadowing js_1_console.js:95
 - (anonymous) js_1_console.js:101
 - XHR/fetch Breakpoints

2. Diff Scope

```
1 let b = 3000;
2 {
3   let a = 10;
4   const b = 20;
5   var c = 30;
6   console.log(a);
7   console.log(b);
8   console.log(c);
9 }
10 console.log(b);
11
```

6_practice.js

- ☒ console.log(c);

Scope

- Block
 - a: 10
 - b: 20
- Script
 - b: 3000
- Global
 - alert: f alert()
 - atob: f atob()
 - blur: f blur()
 - btoa: f btoa()
 - c: 30
 - caches: CacheStorage {}